

DocuMind AI

RAG 기반 다중 문서 질의응답 AI 서비스

개발 기간

2026.05 ~ 2026.07

프로젝트 유형

개인 프로젝트

TECH STACK

Python · LangChain · FAISS · BM25 · PyMuPDF
Streamlit · Groq · HuggingFace · RAGAS · Tesseract

목차

01 프로젝트 개요

02 문제 정의

03 시스템 아키텍처

04 기술스택 & 선택근거

05 독립 변수 실험

06 트러블슈팅

07 성능 벤치마크

08 시연화면

09 한계점 & 개선방향

10 배운 점 & 성장

01 프로젝트 개요

다양한 문서(PDF, DOCX, XLSX, PPTX)를 업로드하고 자연어 질문에 대해 문서 기반 답변과 출처를 제공하는 RAG 기반 AI 서비스

개발 동기

문제

- 대용량 문서에서 원하는 정보를 빠르게 탐색하기 어려움
- 키워드 기반 검색은 문맥 이해에 한계
- 일반 LLM은 할루시네이션(허위 정보 생성) 위험 존재
- 문서 출처를 확인할 수 없어 답변의 신뢰성 검증이 어려움

목표

- 문서 기반 RAG 질의응답 서비스 개발
- 답변 근거와 출처 페이지를 자동 제공하여 신뢰성 향상
- 스캔 PDF를 포함한 다양한 문서 형식 지원

핵심 기능



다중 문서 지원

PDF · DOCX · XLSX · PPTX



문서 기반 질의응답

RAG 기반 검색 및 답변 생성



출처 페이지 제공

답변 근거 페이지 즉시 제공



답변 모드

문서 전용 / AI 혼합



OCR 자동 처리

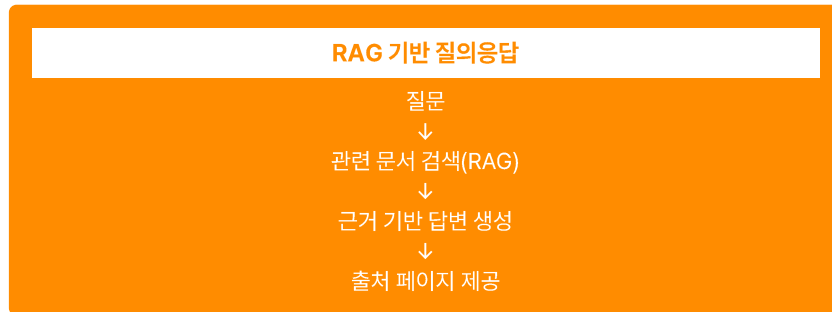
이미지 기반 PDF 자동 인식

02 문제 정의

Before — 기존 방식



After — DocuMind



Q: "이 계약서의 위약금 조항은?"



기존 LLM

일반적인 위약금 조항 생성 / 문서와 무관 / 출처 없음



DocuMind

계약서 17페이지 내용 기반 답변 / 출처(Page 17) 자동 제공

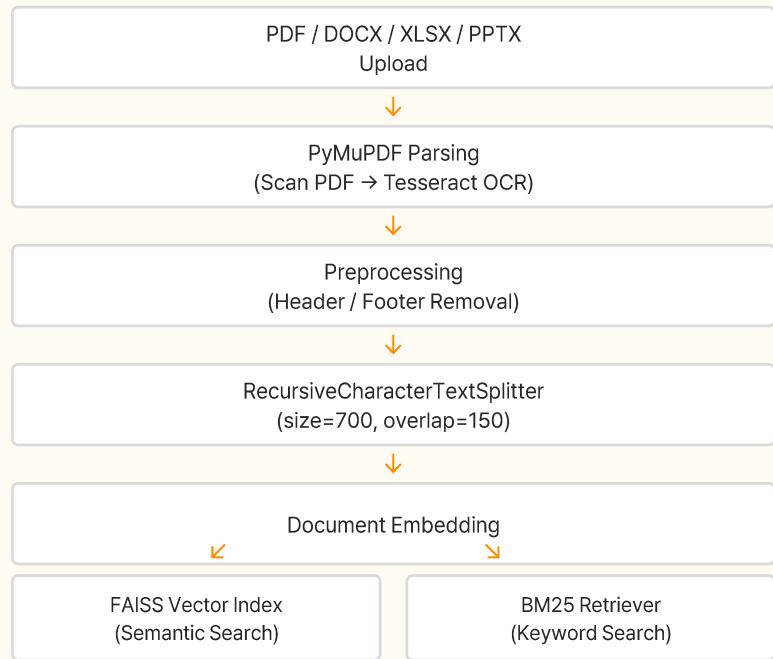
RAG 해결 흐름



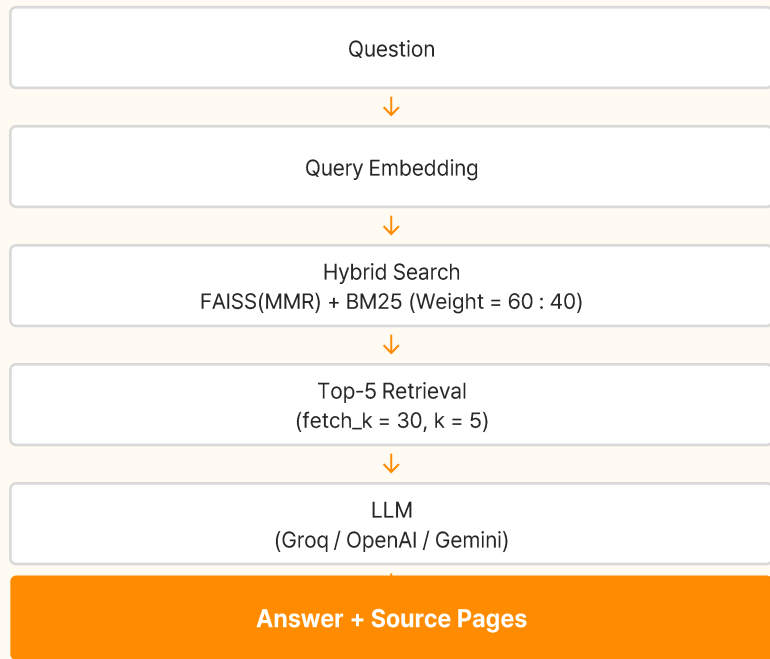
03 시스템 아키텍처

서비스 LLM: llama-3.3-70b | 임베딩: MiniLM-L12-v2 | 검색: FAISS 60% + BM25 40% | 청크: 700/150

Indexing Pipeline (문서 업로드 시 1회 수행)



Query Pipeline (질문마다 수행)



04 기술스택 & 선택근거

Python

AI/ML 생태계 표준 언어로 LangChain, FAISS, PyTorch 등 다양한 라이브러리 지원

LangChain

RAG 파이프라인 구성, 문서 로딩·청킹·검색·프롬프트 체인을 통합 관리

PyMuPDF

PDF 텍스트 추출 및 블록 좌표 기반 읽기 순서 복원, 스캔 PDF 처리 연계

FAISS + BM25

의미 검색과 키워드 검색을 결합한 Hybrid Search로 검색 정확도 향상

Groq

고속 추론 API 제공, 오픈소스 LLM을 낮은 지연시간으로 활용 가능

HuggingFace

다양한 임베딩 모델 제공 및 로컬 임베딩 지원

Streamlit

Python 기반으로 빠른 웹 UI 구현 및 캐싱 기능 지원

RAGAS

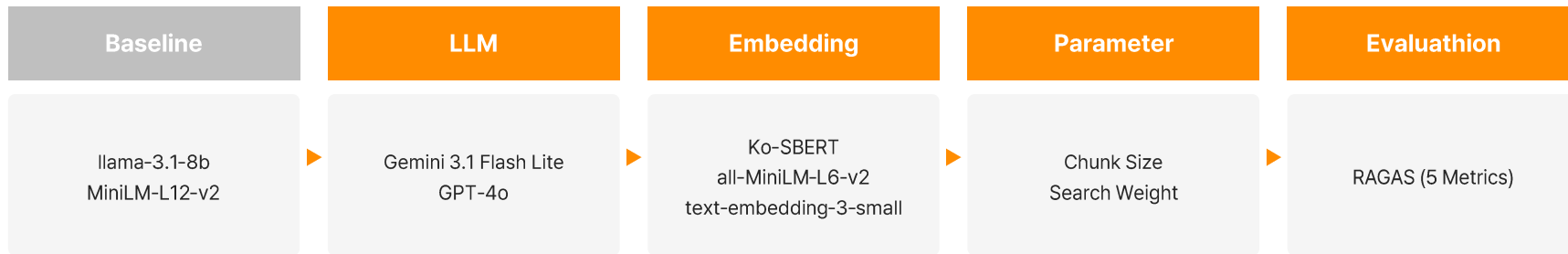
RAG 전용 평가 프레임워크로 Faithfulness, Context Precision 등 정량 평가 지원

Tesseract

스캔 PDF의 텍스트 추출을 위한 오픈소스 OCR 엔진

05 성능 최적화 실험 설계

한 번에 하나의 변수만 변경



실험 원칙

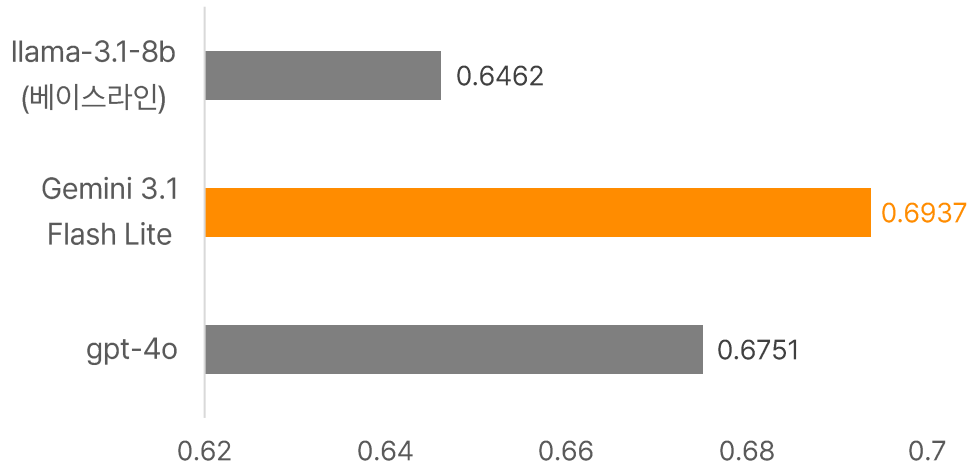
고정 조건

- 한 번에 하나의 변수만 변경
- 나머지 조건 동일 유지
- 변수별 성능 영향 분석

평가 설정

- Judge Model : GPT- 4o-mini
- Evaluation : RAGAS (5 Metrics)
- Dataset : 20 QA
- Rrtrieval : k = 5, fetch_k = 30

06-1 성능 벤치마크 - LLM 비교



평가 조건

Judge Model : GPT-4o-mini

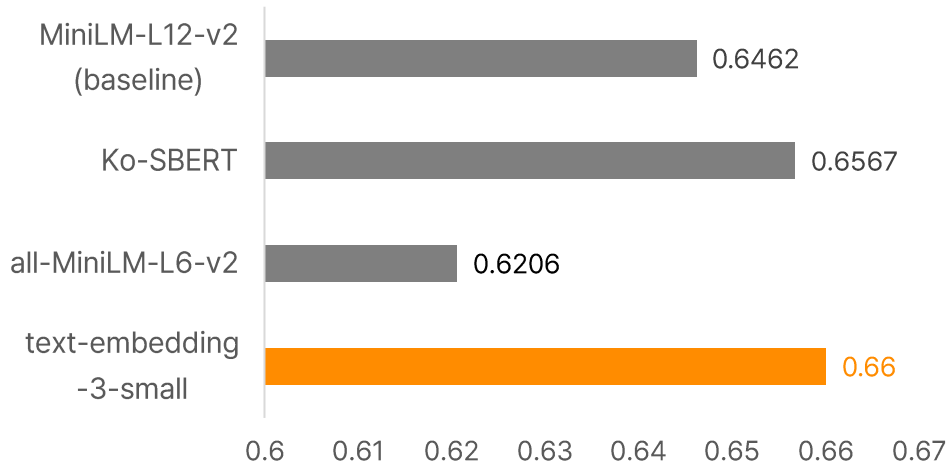
Dataset : 20 QA

Evaluation : RAGAS (5 Metrics)

핵심 인사이트

- 무료 모델 Gemini 3.1 Flash Lite가 유료 gpt-4o보다 높은 평균 점수 (0.6937 vs 0.6751)
- Gemini는 Context Recall 1.000 — 관련 문서를 빠짐없이 검색
- gpt-4o는 Factual Correctness 0.787로 사실 정확도 1위
- llama-3.1-8b는 Answer Relevancy 0.445로 가장 낮음 — 답변 품질 한계 확인

06-2 성능 벤치마크 – Embedding 비교



평가 조건

LLM : llama-3.1-8b (baseline)

Judge Model : GPT-4o-mini

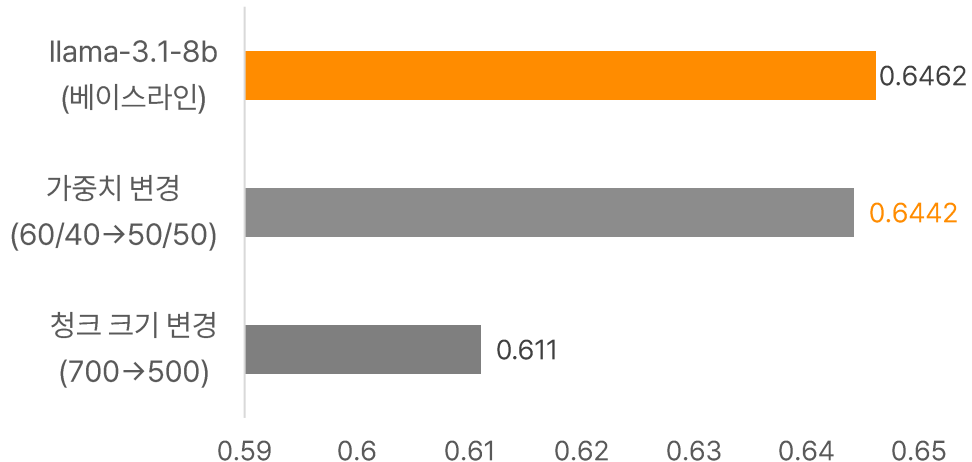
Dataset : 20 QA

Evaluation : RAGAS (5 Metrics)

핵심 인사이트

- text-embedding-3-small이 평균 0.66으로 가장 높은 성능
- Ko-SBERT는 평균 0.6567로 베이스라인(0.6462) 대비 소폭 향상
- all-MiniLM-L6-v2는 평균 0.6206으로 가장 낮음 — Context Precision 0.277로 특히 낮음

06-3 성능 벤치마크 - 검색 파라미터 비교



평가 조건

LLM: llama-3.1-8b
Embedding: MiniLM-L12-v2
Judge Model: GPT-4o-mini
Dataset : 20 QA
Evaluation : RAGAS (5 Metrics)

핵심 인사이트

- 베이스라인(FAISS 60:40)이 균등 가중치(50:50)보다 높은 성능 → 의미 검색 비중이 더 유리
- chunk_size 500은 베이스라인(700) 대비 성능 하락 → 청크가 너무 작으면 문맥 손실
- 베이스라인 파라미터(FAISS 60% + chunk 700)가 최적으로 확인

06-4 성능 벤치마크 - 최종 조합

Best Configuration

LLM

Gemini 3.1 Flash Lite

임베딩

text-embedding-3-small

가중치

FAISS 60% + BM25 40%

최종 조합 평균 점수

0.6406

핵심 인사이트

개별 최고 컴포넌트 조합(0.6406)이 베이스라인(0.6462)보다 낮은 성능

추정 원인

서로 다른 컴포넌트를 조합할 경우 검색 결과와 생성 모델 간 상호작용이 달라져, 개별 최고 성능이 전체 최고 성능으로 이어지지 않음
(특정 LLM과 임베딩 벡터 공간의 표현 방식 간 궁합 불일치 가능성)

→ 통합 테스트와 전체 균형이 중요

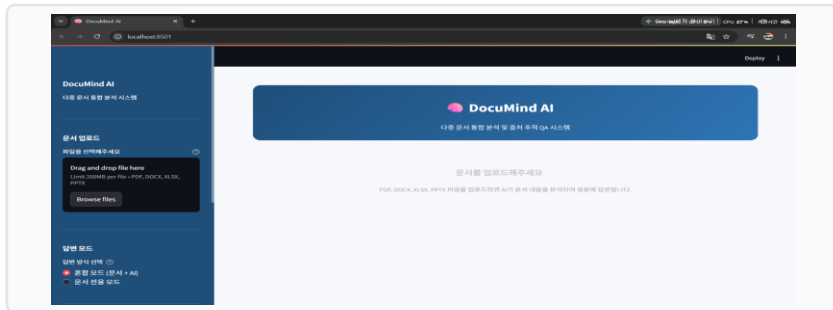
07-1 트러블슈팅 - 파싱·데이터 처리 및 평가 환경

	문제	원인	해결	결과
#1	PyMuPDF4LLM ONNX 오류	pymupdf4llm 적용 과정에서 ONNX Runtime 및 PyMuPDF 버전 충돌	fitz Blocks 모드로 전환 후 좌표 기반 읽기 순서 정렬 직접 구현	안정적 파싱 및 다단 레이아웃 대응
#2	헤더푸터 노이즈	페이지 상·하단 영역을 일괄 필터링할 경우 본문까지 제거되는 문제 발생	반복 출현 텍스트 패턴을 감지하여 헤더·푸터 제거 방식으로 변경	본문 손실을 줄이면서 반복 노이즈 제거
#3	RAGAS 설치 오류	langchain-core 버전 충돌 (0.3.63 vs 1.4.x)	앱 환경과 RAGAS 평가 환경을 Conda 환경으로 분리	의존성 충돌 해결 및 평가 환경 독립성 확보
#4	RAGAS TPM 한도	OpenAI API의 분당 토큰 처리량(TPM) 제한	평가 샘플 간 3초 딜레이 적용	API Rate Limit 발생 완화 및 안정적인 평가 수행
#5	Judge 모델 비용 부담	GPT-4o를 반복 호출하는 평가 구조에서 비용 부담 발생	GPT-4o-mini로 Judge 모델 변경	GPT-4o-mini로 Judge 모델 변경

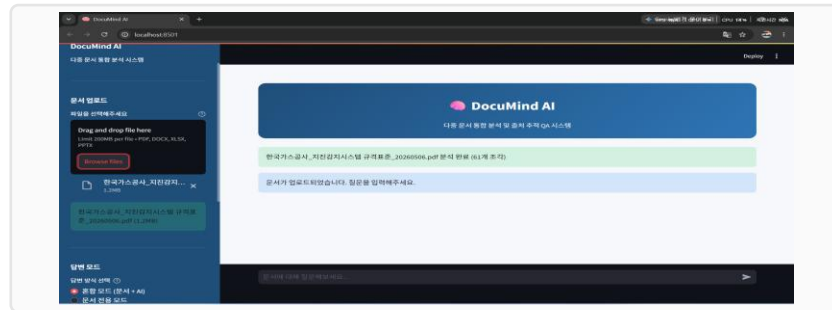
07-2 트러블슈팅 - LLM·RAG 최적화

	문제	원인	해결	결과
#6	Groq 70B 토큰 한도	Groq 무료 API의 TPM 제한으로 llama-3.3-70b 실험 중 토큰 한도 초과	실험 모델을 llama-3.1-8b로 변경	벤치마크 실험 정상 진행
#7	Gemini 2.5 접근 제한	사용 환경에서 Gemini 2.5 모델 접근 제한 발생	사용 가능한 Gemini 3.1 Flash Lite로 변경	LLM 비교 실험 정상 진행
#8	검색 결과 증가에 따른 토큰 초과	k=7 설정으로 검색된 청크 7개가 LLM 입력에 포함되어 토큰 한도 초과	k=7 → k=5로 조정	LLM 입력 토큰 문제 해결
#9	Chunk Size 증가에 따른 토큰 초과	chunk_size=1000 설정으로 검색 문맥의 입력 크기 증가	chunk_size=1000 → 700으로 조정	토큰 초과 문제 완화 및 안정적인 응답 생성

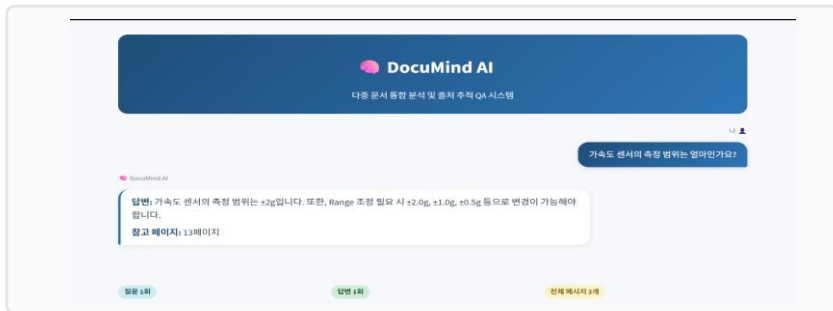
08 시연화면



① 초기 화면 — 다중 파일 업로드 지원



② 업로드 완료 — 문서 자동 분석 및 체크 처리



③ 문서 전용 모드 — 답변 + 출처 페이지 자동 표시



④ 혼합 모드 — 문서 외 질문 시 AI 자체 지식 보완

09 한계점 & 개선방향

한계	원인	개선방향
응답 시간 정량 측정 부족	API 호출 및 대기 시간이 포함되어 순수 파이프라인 성능 비교에 한계	문서 검색·LLM 생성 등 단계별 응답 시간 측정 및 최적화
다양한 도메인 검증 부족	특정 문서 중심으로 평가하여 다양한 분야의 일반화 성능 검증 부족	법률·금융·기술 등 다양한 도메인 문서로 추가 평가
Re-ranking 미적용	검색된 문서를 추가로 재순위화하는 단계가 없어 검색 정확도 향상 여지 존재	Cross-Encoder 기반 Re-ranking 적용
로컬 Vector DB 확장성 한계	현재 FAISS 기반 구조는 대규모 문서 및 서비스 확장 시 관리·확장성에 한계 가능	Qdrant 등 Persistent Vector DB 도입 검토

앞으로 학습·적용할 기술 : Hybrid Search 고도화 · Cross-Encoder Re-ranking · Agentic RAG · Graph RAG

10 배운점 & 성장

기술적 성장

RAG 파이프라인 전체 설계 및 구현

LangChain 기반 FAISS + BM25 앙상블 검색 구현

RAGAS 기반 정량 평가 설계 및 실행

다양한 LLM / 임베딩 API 연동 (Groq, OpenAI, Gemini, HuggingFace)

PyMuPDF + Tesseract OCR 파이프라인 구현

핵심 교훈

RAG 시스템의 성능은 LLM 자체보다 검색 품질·데이터셋 품질·평가 방법이 더 큰 영향을 미치며, 개별 최고 컴포넌트의 조합이 전체 최고 성능을 보장하지 않는다는 것을 직접 실험으로 경험

프로젝트 경험

실험 설계 → 수집 → 채점 → 분석 전체 사이클 경험

트러블슈팅을 통한 문제 해결 능력

무료/유료 API 비용 최적화 경험

Thank You

RAG 기반 다중 문서 질의응답 AI 서비스